

53 - [File upload](#)

Ще се научим как да прикачваме файл (за снимка).

File Upload in ASP.NET Core MVC



Данните ще бъдат записани в БД, а картинката в сървъра в `wwwroot->images`.

До момента, създаването на нов служител става в Create View, пряко обвързан с модел Employee.

```
Create.cshtml Employee.cs ApplicationDbContext...elSnapshot.cs dbo._EFMigrat...History [Data]
1 @model Employee
2 [inject] IEmployeeRepository _employeeRepository
3
4 @{
5     ViewBag.Title = "Create Employee";
6 }
7 <form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
8     <div class="form-group row">
9         <label asp-for="Name" class="col-sm-2 col-form-label"></label>
10        <div class="col-sm-10">
11            <input asp-for="Name" class="form-control" placeholder="Name" />
12            <span asp-validation-for="Name" class="text-danger"></span>
13        </div>
14    </div>
```

За картинка обаче, ще е необходимо да създадем друг ViewModel, който е вид разширение към основния View, специално за целта, по същият начин както създадохме HomeDetailsViewModel.cs, за да допълним към клас Employee + PageTitle.

```
HomeDetailsViewModel.cs Create.cshtml Employee.cs ApplicationDbContext...
EmployeeManagement EmployeeManagement.ViewModels.Home
1 using EmployeeManagement.Models;
2
3 namespace EmployeeManagement.ViewModels
4 {
5     5 references
6     public class HomeDetailsViewModel
7     {
8         5 references
9         public Employee Employee { get; set; }
10        1 reference
11        public string PageTitle { get; set; }
12    }
13 }
```

Папка ViewModels ->Add->Class->EmployeeCreateViewModel.

Това е необходимо, за да имаме достъп до картинката от сървъра чрез ModelBinding,

използвайки тип IFormFile.

Класът, който ще направим, ще е много сходен с Employee, така че копираме наготово от него съдържанието, за да го изменим леко, като отстраним излишния id, вкараме необходимите namespace и изменим името вместо PhotoPath->Photo и тип вместо string ->IFormFile, изискващ

```
public class EmployeeCreateViewModel
{
    [Required]
    [MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$",
        ErrorMessage = "Invalid email format")]
    [Display(Name = "Office Email")]
    public string Email { get; set; }
    [Required]
    public Dept? Department { get; set; }
    public IFormFile Photo { get; set; }
}

using Microsoft.AspNetCore.Http;
Microsoft.AspNetCore.Http.IFormFile
Generate type 'IFormFile'
```

CS0246 The type or namespace name 'IFormFile' could not be found (are you missing a using directive or an assembly reference?)

```
using EmployeeManagement.Models;
using Microsoft.AspNetCore.Http;
```

IFormFile съдържа много полезни properties, изброени по-надолу.

Ако решим да не създаваме отделен ViewModel, а просто да сменим формата на IFormFile, ще трябва да сменим navigation property, понеже IFormFile is a complex object, което ще усложни обработката въобще, освен това, не искаме цялата тази информация да се съхранява в БД, а само името на файла. Затова в Employee оставяме тип string PhotoPath, която да се съхранява в таблица БД, а отделно правим ViewModel, чрез който да прикачим файл към сървъра като IFormFile Photo.

File Upload in ASP.NET Core MVC

```
public class EmployeeCreateViewModel
{
    // Other Employee Properties
    public IFormFile Photo { get; set; }
}

public interface IFormFile
{
    string ContentType { get; }
    string ContentDisposition { get; }
    IDictionary Headers { get; }
    long Length { get; }
    string Name { get; }
    string FileName { get; }
    Stream OpenReadStream();
    void CopyTo(Stream target);
    Task CopyToAsync(Stream target, CancellationToken cancellationToken = null);
}
```

```

Employee.cs Create.cshtml
@model Employee
@inject IRepository _employeeRepository

@{
    ViewBag.Title = "Create Employee";
}

<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group row">
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <div class="col-sm-10">
            Total Employees Count = @_employeeRepository.GetAllEmployee().Count()
        </div>
    </div>
</form>

```

В Create.cshtml, правим връзка към model вместо Employee, да бъде към по-сложния тип EmployeeCreateViewModel, отстраняваме и този излишен ред @inject, както и брояча.

```

Employee.cs Create.cshtml
@model EmployeeCreateViewModel

@{
    ViewBag.Title = "Create Employee";
}

<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group row">
        <div class="col-sm-10"></div>
    </div>
</form>

```

Допълваме вида на формата `<form enctype="multipart/form-data">` и копираме и изменяме кода на един ред, за да бъде нагоден за прикачването.

```

ViewModels Create.cshtml @X
@model EmployeeCreateViewModel

@{
    ViewBag.Title = "Create Employee";
}

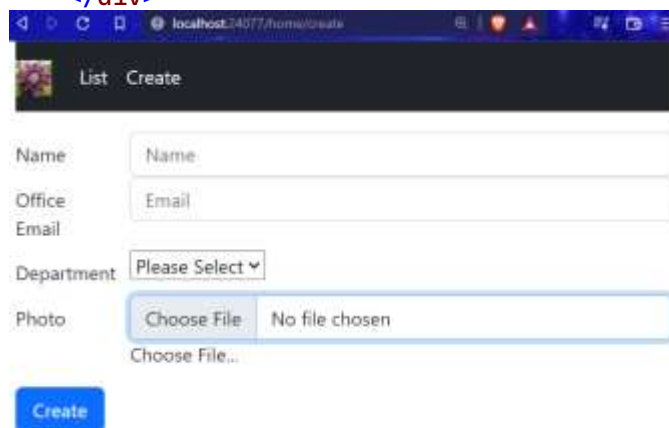
<form enctype="multipart/form-data" asp-controller="Home" asp-action="Create" method="post">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Photo" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <div class="custom-file">
                <input asp-for="Photo" class="form-control custom-file-input" type="file" />
                <label class="custom-file-label">Choose File...</label>
            </div>
        </div>
    </div>
    <div asp-validation-summary="All" class="text-danger"></div>
</form>

```

```

<div class="form-group row">
    <label asp-for="Photo" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
        <div class="custom-file">
            <input asp-for="Photo" class="form-control custom-file-input" type="file" />
            <label class="custom-file-label">Choose File...</label>
        </div>
    </div>
</div>
</div>

```



Никъде не сме уточнили input element, type file.

Това идва от съвпаденията на

```
public IFormFile Photo { get; set; }
```

```
<=>
```

```

<input asp-for="Photo"
    <div class="form-group row">
        <label class="col-sm-2 col-form-label" for="Photo">Photo</label>
        <div class="col-sm-10">
            <div class="custom-file">
                <input class="form-control custom-file-input" type="file" id="Photo" name="Photo" />
                <label class="custom-file-label">Choose File...</label>
            </div>
        </div>
    </div>
</div>

```


Photo

Choose File Mary.jpg
Browse Mary.jpg

```
@if (IsSectionDefined("Scripts"))  
{  
    @RenderSection("Scripts", required: true)  
}  
</div>  
</body>
```

В този урок не се изписва името при избирането на файла, а едва, когато се посочи с мишката в label на полето за file upload control. При моята версия обаче, това става автоматично.

В _Layout бяхме дефинирали, че ако дъното на страницата съдържа секция Scripts, тя да се изпълни. В този урок я извикват за JQuery, за да се изпише името на файла, когато е uploaded, което в моя случай е излишно.

```
<div class="form-group row">  
    <label asp-for="Photo" class="col-sm-2 col-form-label"></label>  
    <div class="col-sm-10">  
        <div class="custom-file">  
            <input asp-for="Photo" class="form-control custom-file-input" />  
            <label class="custom-file-label">Choose File...</label>  
        </div>  
    </div>  
</div>  
<div asp-validation-summary="All" class="text-danger"></div>  
<div class="form-group row">  
    <div class="col-sm-10">  
        <button type="submit" class="btn btn-primary">Create</button>  
    </div>  
</div>  
@section Scripts {  
    <script>  
        $(document).ready(function () {  
            $('.custom-file-input').on("change", function () {  
                var fileName = $(this).val().split("\\").pop();  
                $(this).next('.custom-file-label').html(fileName);  
            });  
        });  
    </script>  
}
```

```
@section Scripts {  
    <script>  
        $(document).ready(function () {  
            $('.custom-file-input').on("change", function () {  
                var fileName = $(this).val().split("\\").pop();  
                $(this).next('.custom-file-label').html(fileName);  
            });  
        });  
    </script>  
}
```

При натискане на бутон Create, [HttpPost] обработва заявката, като вече имаме по-сложен модел от Employee -> EmployeeCreateViewModel

```

[HttpPost]
0 references
public IActionResult Create(Employee employee) //receive the employee object
{
    if (ModelState.IsValid)
    {
        Employee newEmployee = _employeeRepository.Add(employee);
        // return RedirectToAction("details", new { id = newEmployee.Id });
    }
    return View();
}

```

За целта, за да може при изборът да се запази в папка wwwroot/images, ни трябва IHostingEnvironment:

```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private
    0 references
    public HomeController(IEmployeeRepository employeeRepository,
        IHostingEnvironment hostingEnvironment)

```

soft.AspNetCore.Hosting;
 soft.Extensions.Hosting;
 'IHostingEnvironment'
 e 'IHostingEnvironment'
 g

CS0246 The type or namespace name 'IHostingEnvironment' could not be found (are you missing a using directive or an assembly reference?)
 Lines 2 to 3
 using EmployeeManagement.ViewModels;
 using Microsoft.AspNetCore.Hosting;
 using Microsoft.AspNetCore.Mvc;

Създаваме частно поле с променлива

```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    private readonly IHostingEnvironment hostingEnvironment; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository,
        IHostingEnvironment hostingEnvironment)
    {
        _employeeRepository = employeeRepository;
        this.hostingEnvironment = hostingEnvironment;
    }
}

```

извикваме променливата със свойство WebRootPath, което предоставя абсолютно физическо пространство в wwwroot. Трябва да комбинираме неговия адрес с името на прикачения файл. Искаме всяка картинка да бъде с уникално име, а ако се въведат две с еднакво – последната да презапише предходната. За целта се използва **Guid = Global Unique Identifier**. За да присвоим стринговата комбинация на пътя от папката+името на променливата, използваме CopyTo method:

```

HomeController.cs | Create.cshtml
EmployeeManagement.Controllers.HomeController | Create(EmployeeCreateViewModel model)
[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;
        if (model.Photo != null) uploaded
        {
            var name sys class .string method wwwroot/ folder
            string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "images");
            var to store = Global Unique Identifier (generated number) as string + _ + name of uploaded photo
            uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
            sys class
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
        } to create it to the server
        Employee newEmployee = new Employee
        {
            Name = model.Name,
            Email = model.Email,
            Department = model.Department,
            PhotoPath = uniqueFileName
        };
        _employeeRepository.Add(newEmployee);
        return RedirectToAction("details", new { id = newEmployee.Id });
    }
}
FileMode.Create));
C# FileStream.Create() (+ 13 overloads)
Initializes a new instance of the FileStream class with the specified path and creation mode.
FileMode.Create = 2
Specifies that the operating system should create a new file. If the file already exists, it will be overwritten. This requires System.Security.Permissions.FileIOPermissionAccess.Write permission. FileMode.Create is equivalent to requesting that if the file does not exist, use FileMode.CreateNew; otherwise, use FileMode.Truncate. If the file already exists but is a hidden file, an UnauthorizedAccessException exception is thrown.

```

Оказва се, че трябва да добавя наклонена черта за адреса на папка /images, което ми направи печатление като липса още при писането, но в старите версии е работело без нея.



```

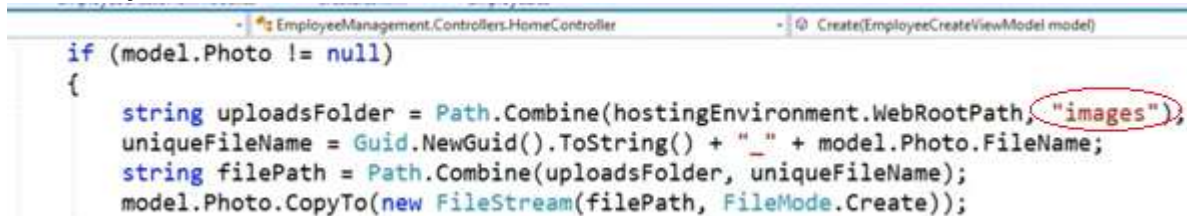
[HttpPost]
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;
        if (model.Photo != null)
        {
            string uploadFolder =
            Path.Combine(hostingEnvironment.WebRootPath + "/images");
            uniqueFileName = Guid.NewGuid().ToString() + '_' +
            model.Photo.FileName;
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath,
            FileMode.Create));
        }
        Employee newEmployee = new Employee
        {

```

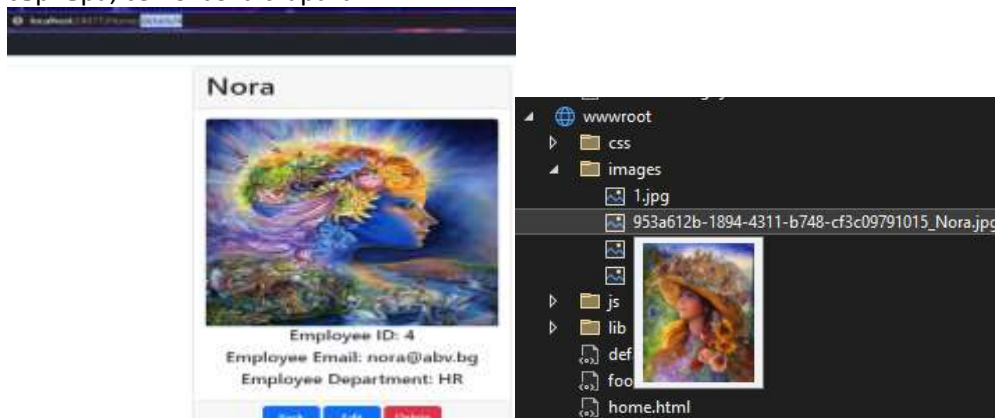
```

        Name = model.Name,
        Email = model.Email,
        Department = model.Department,
        PhotoPath = uniqueFileName
    };
    _employeeRepository.Add(newEmployee);
    return RedirectToAction("details", new { id = newEmployee.Id });
}
return View();
}

```



Когато успява да се създаде новият Employee обаче, въпреки, че картинката е записана в сървъра, се показва старата.



Това е така, защото сме hard code-нали друга по подразбиране.

```

<div class="card-body text-center">
    

    <h4>Employee ID: 4</h4>
    <h4>Employee Email: nora@abv.bg </h4>
    <h4>Employee Department: HR </h4>
</div>
<div class="card-footer text-center">
    <a class="btn btn-primary" href="/">Back</a>
    <a href="#" class="btn btn-primary">Edit</a>
    <a href="#" class="btn btn-danger">Delete</a>
</div>

```

За да го променим, трябва да подадем динамично данните на новата картинка.


```

Employee.cs    Details.cshtml*    HomeDetailsViewModel.cs
1  @model HomeDetailsViewModel
2  @{
3      ViewBag.Title = "Employee Details";
4      var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
5                                     if is not NULL          NULL
6      <div class="row justify-content-center m-1">
7          <div class="col-sm-4">
8              <div class="card">
9                  <div class="card-header">
10                     <h1>@Model.Employee.Name</h1>
11                 </div>
12
13                 <div class="card-body text-center">
14                     
15
16                     <h4>Employee ID: @Model.Employee.Id</h4>
17                     <h4>Employee Email: @Model.Employee.Email </h4>
18                     <h4>Employee Department: @Model.Employee.Department </h4>
19                 </div>

```

Имаме 2 служителя, които директно са вкарани в БД, без те да имат път по същия начин към профилни снимки, затова слагаме опция ??

dbo.Employees [Data] dbo.Employees [Design] Employee.cs Details.cshtml*					
	Id	Name	Email	Department	PhotoPath
	1	Mary	mary@abv.bg	2	NULL
	2	John	john@abv.bg	1	NULL
	4	Nora	nora@abv.bg	1	953a612b-1894-4311-b748-cf3c09791015_Nora.jpg
	NULL	NULL	NULL	NULL	NULL

```

@model HomeDetailsViewModel
@{
    ViewBag.Title = "Employee Details";
    var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
}
<div class="row justify-content-center m-1">
    <div class="col-sm-4">
        <div class="card">
            <div class="card-header">
                <h1>@Model.Employee.Name</h1>
            </div>

            <div class="card-body text-center">
                

```

Леки корекции правим и в Index, където също е hard code, но се използва друг модел като списък, лист IEnumerable

```

IEmployeeRepository.cs | Index.cshtml* | Details.cshtml* | HomeDetailsViewModel.cs
@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">
    @foreach (var employee in Model)
    {
        var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");

        <div class="card m-3">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
            <div class="card-footer text-center">
                <a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
                class="btn btn-primary m-1">View</a>
                <a href="#" class="btn btn-primary m-1">Edit</a>
                <a href="#" class="btn btn-danger m-1">Delete</a>
            </div>
        </div>
    }
</div>

```

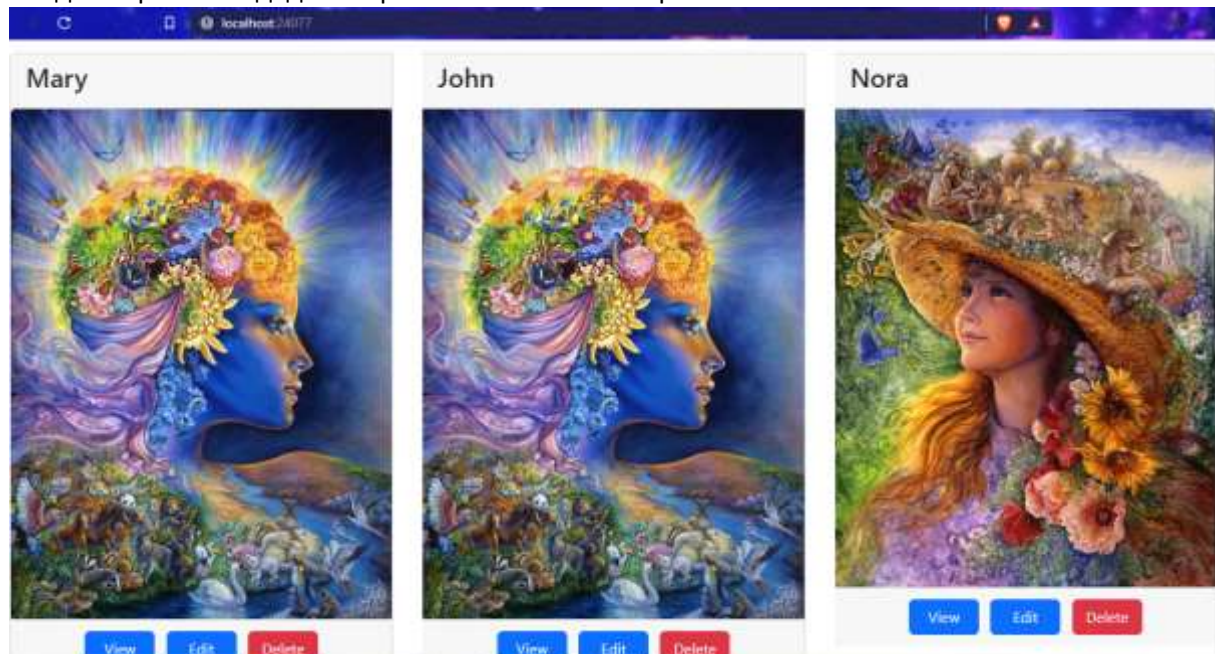
@model IEnumerable<Employee>

```

@{
    ViewBag.Title = "Employee List";
}
<div class="card-group">
    @foreach (var employee in Model)
    {
        var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");
        <div class="card m-3">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
        </div>
    }
</div>

```

И в двата файла създадената photoPath е локална променлива.



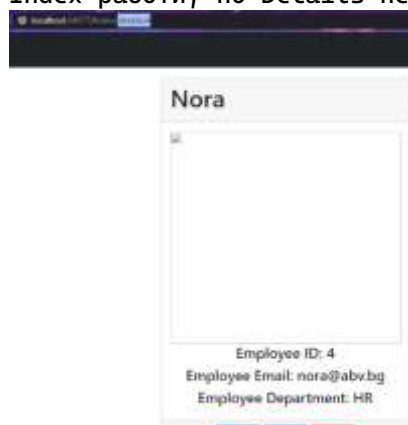
```

<div class="card-header">
  <h3>Mary</h3>
</div>

<div class="card-footer text-center">
  <a class="btn btn-primary m-1" href="/home/details/1">View</a>
  <a href="#" class="btn btn-primary m-1">Edit</a>
  <a href="#" class="btn btn-danger m-1">Delete</a>
</div>
</div>
<div class="card m-3">
  <div class="card-header">
    <h3>John</h3>
  </div>
  
  <div class="card-footer text-center">
    <a class="btn btn-primary m-1" href="/home/details/2">View</a>
    <a href="#" class="btn btn-primary m-1">Edit</a>
    <a href="#" class="btn btn-danger m-1">Delete</a>
  </div>
</div>
<div class="card m-3">
  <div class="card-header">
    <h3>Nora</h3>
  </div>
  

```

Index работи, но Details не – липсва апендикс на версията.



```

<li class="nav-item">
  <a class="nav-link text-light" href="/">List</a>
</li>
<li class="nav-item">
  <a class="nav-link text-light" href="/home/create">Create</a>
</li>
</ul>
</div>
</nav>
<div>
  <div class="row justify-content-center m-1">
<div class="col-sm-4">
  <div class="card">
    <div class="card-header">
      <h1>Nora</h1>
    </div>
    <div class="card-body text-center">
      

```

Използваме image TagHelper for cache busting service.

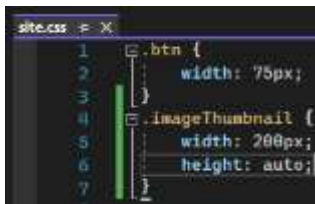
```

<div class="card-body text-center">
  
</div>


```

При което се зарежда и тя със същата валидация.

При Index обаче, всички картинки са с различни големина, затова използваме custom css class.



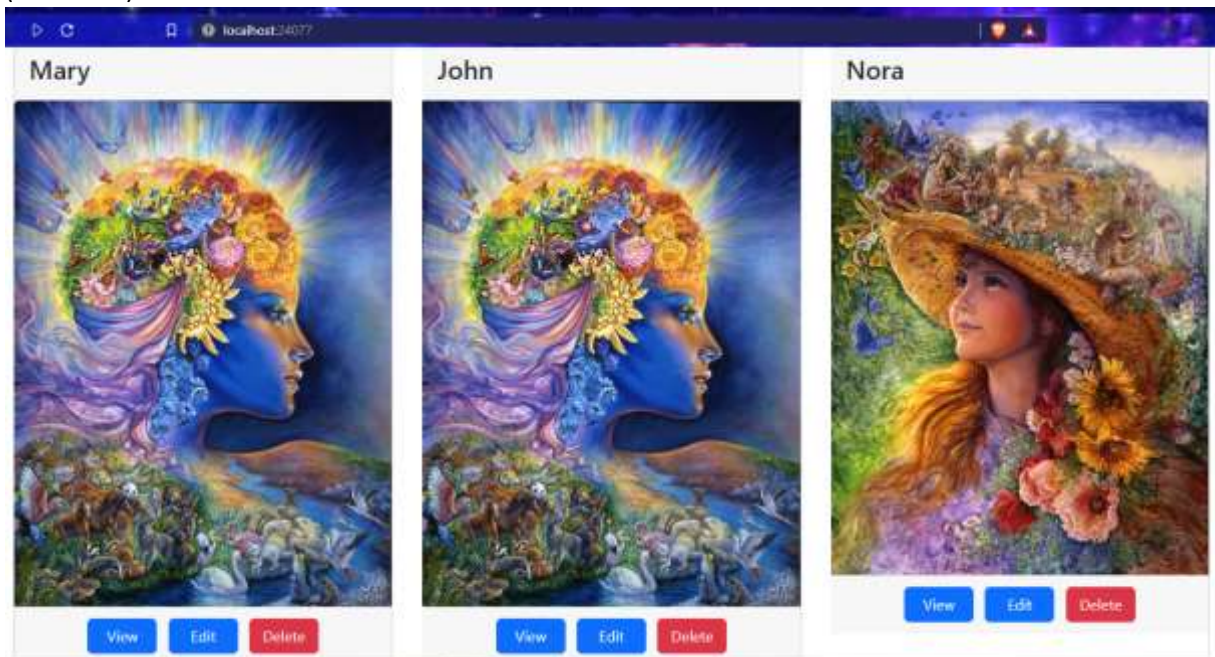
който добавяме в класа на картинката в Index.

```

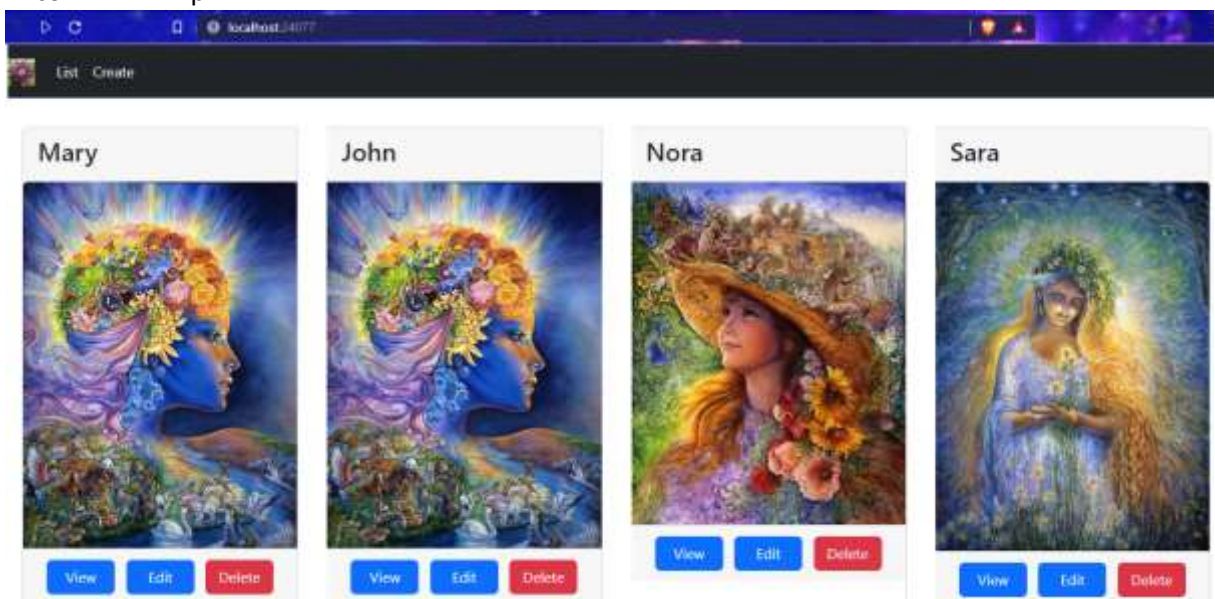
```

Ctrl+R = Reload

(Ctrl+Shift) + R = Hard Reload



Създаваме нов Employee – явно самата картинка е по-къса, защото останалите са с равни височини и ширини.

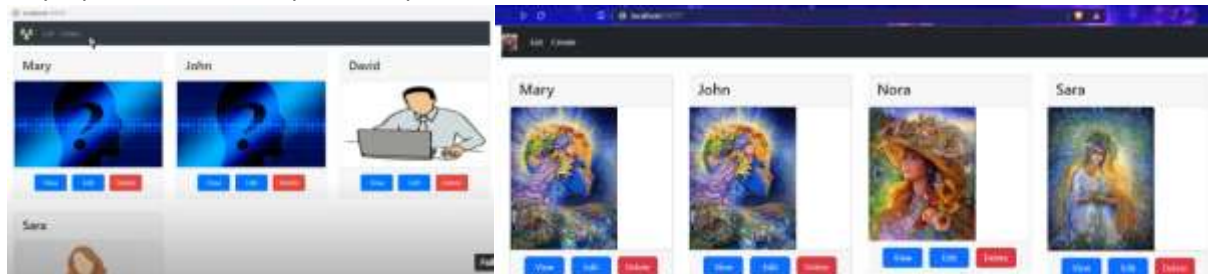


Искаме да има само по максимум 3 картинки на ред. Има няколко начина. Най-бързото и лесно решение е да се сложи максималната ширина да е под 30%:

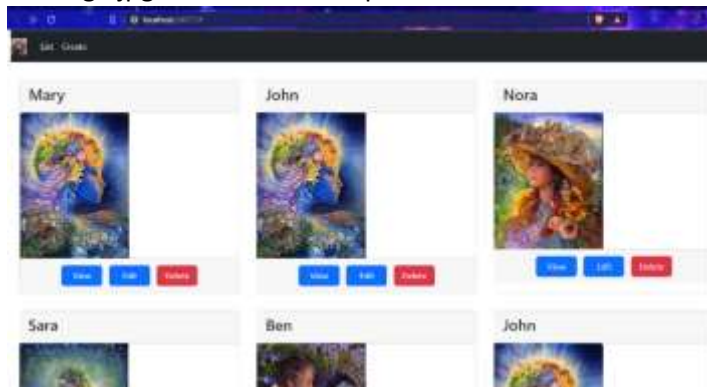

```
@foreach (var employee in Model)
{
    var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");
    <div class="card m-3" style="min-width: 18rem; max-width: 30.5%;">
        <div class="card-header">
```

```
<div class="card m-3" style="min-width: 18rem; max-width: 30.5%;">
```

В туториъла са по 3 на ред, но при мен само си смалиха големините.



При `<div class="card m-3" style="min-width: 20rem; max-width: 30.5%;">` се получават по 3 на моя монитор, също поиграх с `justify`, но са много видове. С демонстративна цел създаваме служител без картинка (без да искам повторих името) – `noimage.jpg` автоматично е присвоена.



В БД се вижда, че е без картинка.

dbo.Employees [Data] dbo.Employees [Design] site.css Employee.cs IEmployeeRepository.cs					
Max Rows: 1000					
	Id	Name	Email	Department	PhotoPath
▶	1	Mary	mary@abv.bg	2	NULL
	2	John	john@abv.bg	1	NULL
	4	Nora	nora@abv.bg	1	953a612b-1894-4311-b748-cf3c09791015_Nora.jpg
	5	Sara	sara@abv.bg	3	b00b44f3-0a95-4223-8c53-3d1c53634262_Sara.jpg
	6	Ben	ben@abv.bg	2	694b5a23-c4c6-4d72-ae9d-e0392eefda44_Ben.jpg
	7	John	john@abv.bg	2	NULL
⚙	NULL	NULL	NULL	NULL	NULL